

PARA LOS EQUIPOS · ENTREGABLES OFICIALES

Plantillas de Entrega

Documentos formales del Proyecto Final

Este documento contiene las 3 plantillas que tu equipo debe entregar el día del Demo Day. Cópialas, llénalas y súbelas al repositorio en las rutas indicadas.

#	Plantilla	Ruta en repo
1	Carátula del proyecto	/ENTREGA.md o /ENTREGA.docx
2	Documento de arquitectura	/docs/arquitectura.md o /docs/arquitectura.docx
3	Documento de negocio	/business/negocio.md o /business/negocio.docx

Cómo usar este documento

1. Cada plantilla está separada por una página de portada para que las puedas convertir fácilmente en archivos individuales.
2. Las cajas amarillas son espacios para llenar. Reemplaza el texto con tu contenido y borra los placeholders.
3. Las instrucciones en gris cursiva ("[Instrucción: ...]") deben borrarse antes de entregar.
4. Los títulos, encabezados y estructura no se modifican.
5. Si una sección no aplica a tu proyecto, escríbelo explícitamente: "No aplica porque...".
6. Pueden entregar en formato Word (.docx) o Markdown (.md), lo que prefieran.

Identificación del equipo y resumen del entregable

Proyecto Final: PgGuardian

Contiene los datos del equipo, el reparto real de roles, los enlaces a los entregables y la declaración obligatoria sobre uso de IA.

Súbelo al repositorio como /ENTREGA.md o /ENTREGA.docx en la raíz.

Datos del proyecto

Nombre del proyecto	PgGuardian
Nombre del equipo	Hi-5
Slogan de su producto	La única defensa que tú necesitarás
Fecha de entrega	Jueves 14 de mayo de 2026

Miembros del equipo

Nombre completo	Matrícula	Rol principal	Email
Luis Alberto Arias Llaguno	00483701	Full Stack e Índices	luis.llaguno@anahuac.mx
Jesús Elias Avila Sauza	00521518	Backend Developer y Estadísticas avanzadas	jesus_avila@anahuac.mx
David Ceballos Mata	00476577	Arquitecto Técnico (Documento de arquitectura) y Observabilidad	david.ceballos24@anahuac.mx
Ana Sofía Ruiz Castillo	00485692	Arquitecta Técnica (Documento de arquitectura) y Memoria	aruiz2604@anahuac.mx
Isabel Ixchel Torres Sánchez	00507838	Analista de Negocio (Documento de negocio) y Bloat/Vacuum	ixchel_torres@anahuac.mx

Repositorio Git

Video demo (3-5 min)

Landing page (si aplica)

<https://github.com/luiselodeon/PgGuardian>

Link a YouTube, Drive, Loom, etc.

Solo si tomaron el bonus

Producto desplegado (si aplica)

<https://pgguardian-frontend.onrender.com>

Resumen ejecutivo

Las PyMEs mexicanas que usan PostgreSQL en producción enfrentan un problema: su base de datos se degrada con el tiempo, pero no tiene un especialista (DBA) que la monitoree. Los desarrolladores, enfocados en sus nuevas funcionalidades no ven el problema hasta que suceden caídas inesperadas o pérdida de transacciones, generalmente a las 3:00 a.m.

PgGuardian es un auditor automático que se conecta de forma segura y solo de lectura a la base de datos del cliente. En menos de 24 horas entrega un reporte claro y priorizado con los problemas encontrados. No se necesita ser un experto en base de datos para entenderlo.

Nuestro usuario objetivo son PyMEs mexicanas de 5 a 50 desarrolladores que tienen PostgreSQL en producción, pero carecen de un DBA dedicado, como candidatos tenemos a sectores de logística o Fintech.

Frente a competidores internacionales como pganalyze o Datadog. PgGuardian se diferencia por tres razones: es accesible para el mercado latinoamericano, tiene soporte nativo en español con documentación legislada a las leyes mexicanas y ofrece una opción self-hosted para Fintechs que no puedan enviar su metadada a servidores externos. No solo somos más baratos sino que somos un producto específicamente para las necesidades y realidades de las empresas de la región.

Cobertura sobre BD demo oficial

BD demo utilizada	TiendaDB
-------------------	----------

Total de problemas plantados	18
------------------------------	----

Problemas detectados correctamente	22
------------------------------------	----

Falsos positivos reportados	0
-----------------------------	---

% de cobertura calculado	100%
--------------------------	------

Declaración de uso de IA

Por qué es obligatoria esta sección

El uso de IA generativa está permitido. Lo que NO está permitido es no declararlo.

Penalización por no declarar: -5 pts en la nota final.

Sé honesto. Es mejor declarar uso amplio que ocultarlo y que se note en el Q&A.

Herramientas de IA utilizadas durante el desarrollo:

- ☒ ChatGPT / GPT-4 / GPT-5
- ☒ Claude (Anthropic)
- ☐ GitHub Copilot

- ☐ Cursor / Windsurf / similar
- ☒ Gemini
- ☐ Otras: _____
-

Áreas donde se usó IA (marquen con [x] las que apliquen):

- ☒ Generación de código (boilerplate, funciones, scripts)
- ☒ Debugging y resolución de errores
- ☒ Generación de queries SQL complejas
- ☒ Documentación técnica
- ☒ Documento de negocio
- ☐ Diseño de arquitectura
- ☐ Generación de datos sintéticos para pruebas
- ☐ Como parte del producto en runtime (ej: PgPilot)
- ☒ Otra: Generación de logo del producto

Descripción detallada del uso de IA en el proyecto:

Usamos Claude (Anthropic) como herramienta principal de asistencia durante el desarrollo. En la fase de detección, nos ayudó a generar el boilerplate inicial de las funciones de conexión a `pg_stat_user_indexes` y `pg_constraint`, pero la lógica de cada detector (umbrales de severidad, filtros para evitar falsos positivos, queries al catálogo) fue diseñada y validada manualmente por el equipo ejecutando cada query contra TiendaDB.

Para los queries SQL complejos (como el cálculo de bloat basado en `pg_class` y el análisis de distribución estadística con `pg_stats`), usamos Claude para entender la documentación de PostgreSQL y generar versiones iniciales que luego corregimos con debugging propio. Gemini se usó para generar el logo del producto. Del documento de negocio en la parte de las entrevistas se utilizó ChatGPT para hacer los resúmenes y transcripts de las mismas, con respecto al análisis competitivo fueron realizados por el equipo sin asistencia de IA.

Checklist de entregables

- ☐ README claro con instrucciones de instalación y uso
- ☒ docker-compose.yml funcional probado en máquina limpia
- ☐ Documento de arquitectura (esta plantilla, sección 2)
- ☐ Documento de negocio (esta plantilla, sección 3)
- ☐ Catálogo de reglas / detectores / anti-patterns implementados
- ☐ Video demo de 3-5 minutos accesible
- ☐ Historial de Git con contribuciones de los 5 miembros
- ☐ Esta carátula completa con todos los datos

Antes de entregar

1. Borren todas las cajas amarillas con el texto reemplazado por el contenido real.
2. Borren todas las instrucciones en gris cursiva (las que están entre corchetes).
3. Verifiquen que todos los enlaces funcionen desde una sesión incógnita / sin login.
4. Hagan que un miembro externo al equipo intente correr el repo siguiendo solo el README.
5. Si encuentran inconsistencias, ajusten antes del Demo Day.

Decisiones técnicas, diagrama y limitaciones del producto

Documento de Arquitectura

¿Qué es esto?

Es el documento técnico que explica cómo está construido el producto y por qué.

Lo evalúa el Criterio 1.2 de la rúbrica (Justificación técnica de decisiones, 8 pts).

Súbelo al repositorio como `/docs/arquitectura.md` o `/docs/arquitectura.docx`.

Extensión recomendada: 3-5 páginas. No más. Documentos de 20 páginas restan puntos por no saber qué es importante.

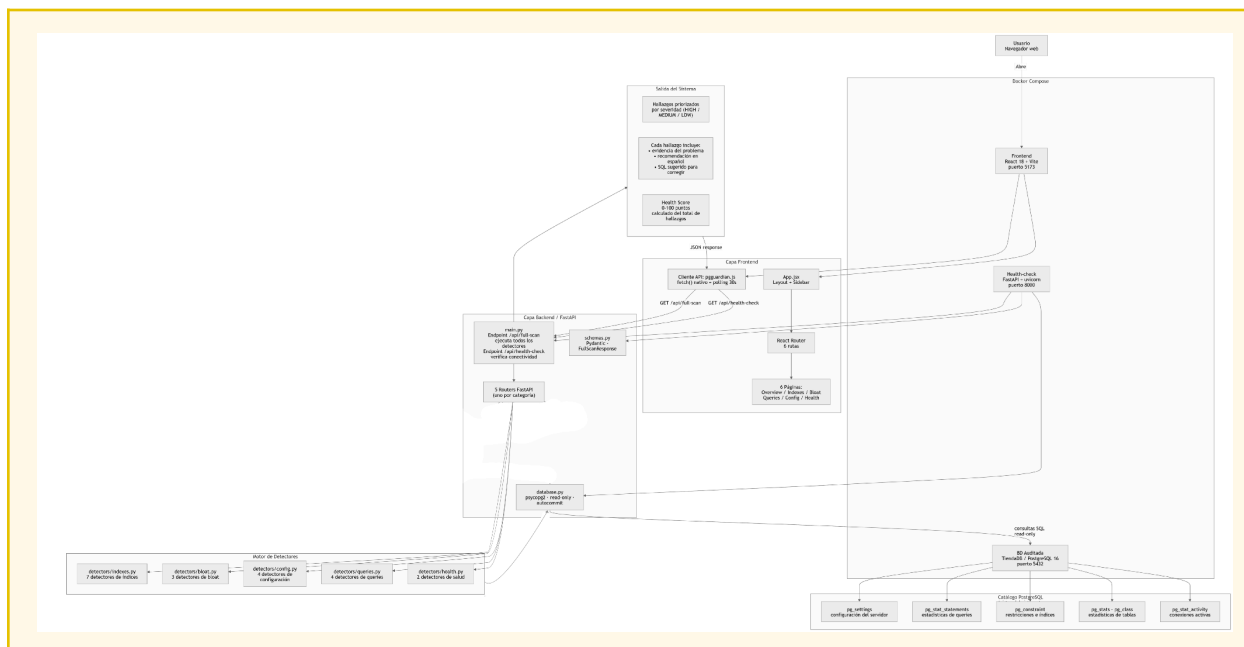
1. Visión general

PgGuardian es un producto B2B de auditoría continua para PostgreSQL. Se conecta de forma read-only a la base de datos del cliente y genera un reporte priorizado con hallazgos en cinco categorías: índices, bloat, queries, configuración y salud general.

El backend está construido en Python/FastAPI con routers separados por categoría. Cada detector es una función independiente que consulta el catálogo del sistema de PostgreSQL (`pg_constraint`, `pg_stats`, `pg_stat_statements`, `pg_settings`, `pg_stat_activity`) sin utilizar ORM ni asumir el schema del cliente. El frontend es una SPA en React/Vite con dashboard de health score, páginas por categoría y tablas de hallazgos con severidad y recomendaciones SQL ejecutables.

El despliegue se realiza con Docker Compose: un contenedor para la BD auditada (TiendaDB), otro para el backend FastAPI, y otro para el frontend React.

2. Diagrama de arquitectura



Descripción del diagrama:

La arquitectura de PgGuardian está organizada en tres servicios orquestados por Docker Compose: el frontend en React 18 + Vite en el puerto 5173, el backend en Python/FastAPI con uvicorn en el puerto 8000 y la base de datos auditada (TiendaDB sobre PostgreSQL 16) en el puerto 5432. El usuario interactúa desde el navegador con el dashboard web, construido con **App.jsx**, React Router y seis páginas: Overview, Bloat, Config, Health, Indexes y Queries. La aplicación ejecuta un escaneo inicial al cargar y mantiene un polling automático cada 30 segundos mediante un toggle en la barra superior. Desde el frontend, el módulo **pgguardian.js** centraliza las llamadas al backend con **fetch()** y expone un endpoint principal: **GET /api/full-scan**.

En la capa backend, **main.py** funciona como punto de entrada de FastAPI. Registra cinco routers (uno por categoría: **bloat**, **config**, **health**, **indexes**, **queries**) mediante **app.include_router()**, e implementa el endpoint agregado **/api/full-scan**, que ejecuta los 25 detectores de forma secuencial. Cada detector se invoca a través de la función **_safe_run()**, que captura excepciones individualmente para que un detector con error no detenga al resto del escaneo. Los detectores viven en el paquete **detectors/** (**bloat.py**, **config.py**, **health.py**, **indexes.py**, **queries.py**) y consultan la base auditada mediante **database.py**, que entrega una conexión **psycopg2** como dependencia de FastAPI usando **Depends()**.

Las consultas se ejecutan sobre vistas del catálogo del sistema PostgreSQL: **pg_settings**, **pg_stat_statements**, **pg_constraint**, **pg_stats**, **pg_class**, **pg_stat_user_tables** y **pg_stat_activity**. Finalmente, **_serialize()** normaliza los resultados (convirtiendo **RealDictRow** a **dict** y **Decimal** a **float**) y **main.py** agrupa los hallazgos por categoría calculando un **health_score** de 0 a 100 basado en penalizaciones diferenciadas por tipo de problema.

3. Componentes principales

Componente / Módulo	Responsabilidad	Tecnología
Docker Compose	Orquesta la BD auditada, backend y frontend en un solo comando	Docker, docker-compose.yml
database.py	Conexión a PostgreSQL como dependencia de FastAPI	psycopg2-binary, Depends()
schemas.py	Define modelos de respuesta genéricos sin usar ORM	Pydantic (BaseModel)
detectors/indexes.py	Implementa 7 detectores relacionados con índices	SQL contra pg_constraint, pg_stats, pg_stat_statements
detectors/bloat.py	Implementa 3 detectores de bloat y mantenimiento	SQL contra pg_stat_user_tables, pg_class
detectors/config.py	7 detectores de configuración	SQL contra pg_settings
detectors/health.py	Implementa 2 detectores de salud general	SQL contra pg_stat_activity, pg_class
detectors/queries.py	6 detectores: seq scans, temp spills, EXPLAIN spills	SQL contra pg_stat_statements + EXPLAIN
api/routers/.py	Expone 5 routers FastAPI, uno por categoría	FastAPI APIRouter
api/main.py	Punto de entrada del backend; configura CORS, /api/full-scan, /api/health-check	FastAPI, _safe_run(), _serialize(), cálculo del health_score
frontend/src/main.jsx	Punto de entrada de React; monta App.jsx con BrowserRouter	React 18, Vite
frontend/src/App.jsx	Define el layout principal con Sidebar, top bar, auto-refresh 30s y React Router	React 18, Vite, react-router-dom
frontend/src/pages/*.jsx	Contiene 6 páginas: Overview y una página por categoría	React, CSS custom, Queries
frontend/src/components/Sidebar	Navegación lateral con badge de hallazgos por categoría	React Router NavLink
frontend/src/components/HealthScore	Círculo SVG con el score 0-100	React, SVG
frontend/src/components/CategoryCard	Tarjeta resumen por categoría con conteo de hallazgos	React, CSS

Componente / Módulo	Responsabilidad	Tecnología
frontend/src/components/DataTable	Tabla con filtros, paginación y SeverityBadge	React
frontend/src/components/SeverityBadge	Badge visual de severidad	React, CSS
frontend/src/components/SQLRecommendation	Bloque con SQL ejecutable y botón "copiar"	React, CSS
frontend/src/api/pgguardian.js	Cliente HTTP con fullScan() + endpoints individuales	fetch() nativo

4. Decisiones técnicas y trade-offs

Estructura sugerida por decisión

Decisión: qué eligieron

Alternativas consideradas: qué más miraron

Por qué eligieron esta: razón principal

Trade-off aceptado: qué pierden a cambio

Decisión 1: No usar ORM

Decisión: Consultar directamente vistas del catálogo y estadísticas internas de PostgreSQL, como `pg_constraint`, `pg_stats`, `pg_stat_statements` y `pg_settings`, usando SQL crudo mediante `psycopg2`.

Alternativas consideradas:

- SQLAlchemy para mapear tablas del cliente.
- Django ORM para estructurar los modelos de datos.

Por qué eligieron esta: PgGuardian no conoce previamente el esquema de la base del cliente. Usar un ORM implicaría asumir una estructura fija de tablas, columnas o relaciones, lo cual no funcionaría si la base cambia de nombres o si se evalúa una versión distinta de TiendaDB. En cambio, las vistas (`pg_constraint`, `pg_stats`) del catálogo de PostgreSQL son generales y existen en cualquier instalación PostgreSQL.

Trade-off aceptado: Perdemos tipado estático y autocompletado del ORM. Lo compensamos con validación Pydantic en la capa de API (`schemas.py`) y pruebas manuales contra TiendaDB, ganando flexibilidad y compatibilidad con distintos esquemas.

Decisión 2: FastAPI como backend

Decisión: Usar FastAPI como backend, con routers separados por categoría de hallazgo.

Alternativas consideradas:

- Flask (conocido por el equipo, más simple) por su simplicidad.
- Django REST Framework por su ecosistema más amplio.

Por qué elegimos esta: FastAPI permite construir endpoints rápidamente, genera documentación automática con Swagger UI en /docs, dando una demo profesional sin esfuerzo adicional de documentación y facilita la validación de respuestas mediante Pydantic. El patrón de Depends() permite inyectar la conexión a PostgreSQL de forma limpia. Además, separar routers por categoría permitió dividir el trabajo del equipo por módulos: índices, bloat, configuración, queries y salud general sin conflictos de merge.

Trade-off aceptado: FastAPI tiene un ecosistema más nuevo y menos estructura integrada que Django, por lo que el equipo tuvo que definir manualmente la organización del proyecto, los routers y el manejo de respuestas. Sin embargo, para el MVP esto no representó una limitación grave, ya que permitió mantener una arquitectura ligera, rápida de integrar y suficiente para exponer los detectores mediante endpoints claros.

Decisión 3: Aislamiento de errores (con `_safe_run_`)

Decisión: Cada detector se ejecuta dentro de una función de aislamiento de errores llamada `_safe_run()`, la cual captura excepciones de forma individual. De esta manera, si un detector falla por un error de permisos, una extensión faltante o un caso no contemplado, los demás detectores continúan ejecutándose y el escaneo completo no se detiene.

Alternativas consideradas:

- Ejecutar todos los detectores en una sola función secuencial sin manejo individual de errores.
- Ejecutar cada detector como un proceso independiente.

Por qué elegimos esta: En una demo en vivo o en una base real, un detector que falle por un edge case no puede tirar el scan completo. El error puede venir de permisos insuficientes, una extensión faltante, una vista no habilitada o una diferencia en la configuración de PostgreSQL. Con `_safe_run()`, el sistema retorna un error controlado para el detector fallido y deja que el resto de los detectores continúe ejecutándose, permitiendo mostrar resultados parciales sin romper toda la auditoría.

Trade-off aceptado: Al capturar errores por detector, existe el riesgo de que algunos fallos queden ocultos si no se reportan correctamente. Por ejemplo, un detector podría fallar por permisos insuficientes, por falta de `pg_stat_statements` o por una extensión no instalada, y aun así el escaneo seguiría mostrando resultados de otros módulos. Para mitigarlo, los errores deben registrarse o devolverse como hallazgos técnicos controlados, indicando qué detector falló y cuál fue la causa, sin afectar el resto del análisis.

Decisión 4: Health score con heurística por tipo de hallazgo

Decisión: Implementamos un Health Score de 0 a 100 que parte de 100 puntos y aplica penalizaciones diferenciadas según el TIPO de problema detectado, no únicamente por su severidad. Ej: una sesión `idle_in_transaction` resta 10 puntos porque bloquea el vacuum, un índice no usado solo resta 3 puntos porque el impacto es menor.

Alternativas consideradas:

- **(Aprobado / Reprobado):** solo decir si la base de datos está "bien" o "mal".
- **Cantidad de problemas:** tratar por igual a todos los problemas, cuando hay unos con más implicaciones que otros.

- **Score ponderado solo por severidad HIGH/MEDIUM/LOW:** no distinguía entre tipos de problema.

Por qué elegimos esta: Queríamos un puntaje que fuera intuitivo. El score refleja esa diferencia entre un problema mayor y uno menos urgente para que el usuario sin DBA entienda dónde está el riesgo real.

Trade-off aceptado: Aceptamos que los pesos son subjetivos. En una versión productiva, los pesos deberían ser configurables por cliente.

Decisión 5: Auto-refresh con polling cada 30 segundos

Decisión: El frontend ejecuta un scan completo automáticamente cada 30 segundos con un toggle para que el usuario pueda desactivarlo desde la barra superior.

Alternativas consideradas:

- Scan manual únicamente (solo botón "Ejecutar Escaneo").
- Server-Sent Events con FastAPI StreamingResponse.

Por qué elegimos esta: El polling es el patrón más simple que cumple el caso de uso de "ver el estado actual sin tener que refrescar manualmente". Server-Sent Events agrega complejidad sin beneficio claro para un dashboard que no necesita actualizaciones en tiempo real.

Trade-off aceptado: Cada cliente abierto en el navegador genera 2 scans por minuto contra la BD auditada, con múltiples usuarios, esto podría saturar la BD.

Decisión 6: Uso de RealDictCursor

Decisión: Decidimos configurar el cursor de conexión de forma que devuelva los resultados de las consultas en forma de diccionarios hacia el API, en lugar de tuplas.

Alternativas consideradas:

- Cursor estándar (Tuplas) y acceder a los datos por índice.
- Usar NamedTupleCursor y acceder por atributo.

Por qué elegimos esta: Puesto que PgGuardian es un consultor automatizado que debe de procesar múltiples tipos de hallazgos, como lo son índices, bloat o de configuración, el usar diccionarios facilita la generación, así como serialización de reportes.

Trade-off aceptado: El usar RealDictCursor representa un consumo de memoria adicional a que si se manejara con tuplas tradicionales, ya que cada fila almacena tanto llave, como valor.

5. Flujo principal de datos

1. El usuario abre **http://localhost:5173** en el navegador, **main.jsx** monta **App.jsx**.
2. **App.jsx** ejecuta automáticamente **runScan()** y configura un **setInterval** cada 30 segundos para repetir el scan mientras **autoRefresh** esté activo.
3. **runScan()** llama a **fullScan()** del módulo **frontend/src/api/pgguardian.js**.
4. **pgguardian.js** hace **fetch()** a <http://localhost:8000/api/full-scan>.
5. FastAPI recibe la petición en **main.py** y obtiene una conexión **psycopg2** mediante **Depends(get_connection)** desde [database.py](#).
6. La función **full_scan()** ejecuta secuencialmente los 25 detectores envolviendo cada uno en **_safe_run()**.
7. Cada detector abre un **cursor** sobre la conexión compartida y ejecuta consultas SQL contra el catálogo del sistema PostgreSQL. Usamos **RealDictCursor** para que los resultados lleguen como diccionarios con los nombres de las columnas.
8. **_serialize()** convierte los **RealDictRow** a **dict** estándar y los valores **Decimal** a **float** para que sean serializables a JSON.
9. **main.py** agrupa los hallazgos en un diccionario **categories** con las 5 categorías. Cada categoría contiene un sub-diccionario **detectors** con los hallazgos de cada detector.
10. Se calcula **penalty** sumando los pesos diferenciados por tipo de hallazgo. (pesos asignados por el equipo).
11. Se calcula **total_findings** sumando los conteos de todos los detectores.
12. FastAPI retorna un **FullScanResponse** en formato JSON con **health_score**, **total_findings**, **categories** y **executed_at**.
13. **App.jsx** guarda la respuesta en el estado **scanData** y actualiza **lastScan** con la fecha actual.
14. **Sidebar.jsx** lee **scanData.categories** y muestra un badge con el conteo de hallazgos por categoría junto a cada enlace de navegación.
15. **Overview.jsx** renderiza **HealthScore** (círculo SVG con el score) y **CategoryCards** (cinco tarjetas con resumen por categoría).
16. El usuario navega a una página específica y ve un **DataTable** con cada hallazgo individual, puede expandir la fila para ver evidencia completa, y copiar el **sql_recommendation**.

6. Limitaciones reconocidas

Tipos de limitaciones que pueden mencionar

- Operativas: cosas que no funcionan en producciones reales (BDs muy grandes, schemas particulares, etc.)
- De alcance: features intencionalmente fuera del MVP
- Técnicas: deuda técnica conocida, casos límite no manejados
- De negocio: segmentos de mercado que no atendemos todavía
- De seguridad/privacidad: información que no protegemos

Limitación 1: Uso de extensiones en detectores

Los detectores **check_table_bloat** y **check_partitioning_candidates** dependen de extensiones como **pgstattuple** y **pg_partman**. En la versión actual, pueden ejecutar **CREATE EXTENSION IF NOT EXISTS**, lo cual requiere privilegios elevados y contradice el principio read-only del producto. En producción real, PgGuardian primero debería validar si la extensión ya existe y, si no está disponible, usar un fallback basado en catálogo o reportar que esa evidencia no puede obtenerse con permisos de solo lectura.

Limitación 2: Sin histórico de scans

El MVP detecta el estado actual de la base de datos, pero todavía no guarda snapshots históricos para comparar tendencias before/after. Esto limita la capacidad de demostrar si la salud de la BD mejora o empeora con el tiempo. En una versión futura se implementaría una BD interna propia de PgGuardian para almacenar scans, hallazgos y health scores históricos.

Limitación 3: Sin autenticación ni autorización

La API no protege endpoints con JWT, OAuth o control de roles. Esto es aceptable para una demo local, pero no sería suficiente para un ambiente productivo. En producción se implementaría autenticación, autorización por usuario y protección de endpoints antes de exponer el servicio.

Limitación 4: Dependencia de pg_stat_statements

Algunos detectores requieren que pg_stat_statements esté habilitado para analizar queries, tiempos acumulados y bloques temporales. Si la extensión no está disponible, estos detectores pueden retornar vacío sin explicar claramente la causa. En una versión futura, PgGuardian debería generar un hallazgo informativo indicando que la extensión no está habilitada y qué visibilidad se pierde.

Limitación 5: Umbrales iniciales no adaptativos

Algunos detectores usan umbrales fijos para clasificar problemas, por ejemplo en configuración de memoria o severidad de hallazgos. Esto funciona para el MVP, pero no considera automáticamente la RAM del servidor, el tamaño de la base ni el tipo de workload. En producción, los umbrales deberían ser configurables y ajustarse con base en el contexto real de cada cliente.

Limitación 6: Sin pool de conexiones ni timeouts avanzados

El MVP ejecuta el escaneo como un flujo controlado, pero todavía no incorpora una estrategia robusta de pool de conexiones, connect_timeout y statement_timeout para escenarios de alta concurrencia o bases muy grandes. En producción, estos controles serían necesarios para evitar scans demasiado largos o saturación accidental de la base auditada.

7. Roadmap técnico (post-MVP)

Próximos 3 meses:

- **Eliminar el uso de CREATE EXTENSION dentro de los detectores.**
Se implementará una validación previa para revisar si extensiones como pgstattuple o pg_partman ya existen. Si no están disponibles, PgGuardian usará consultas alternativas basadas en catálogo o reportará la limitación sin romper el análisis.
- **Agregar una BD interna de PgGuardian para snapshots históricos.**
Esto permitirá almacenar resultados de cada scan, comparar tendencias before/after y mostrar si la salud de la base mejora o empeora con el tiempo.
- **Generar reportes PDF exportables desde el dashboard.**
El objetivo es que los usuarios puedan compartir resultados ejecutivos con líderes técnicos, gerencia o equipos de cumplimiento.
- **Agregar timeouts y control de conexiones.**
Se incorporarán connect_timeout, statement_timeout y una estrategia inicial de pool de conexiones para evitar scans demasiado largos o saturación accidental.

Próximos 6 meses:

- **Alertas configurables por email o Slack.**
PgGuardian enviará notificaciones cuando el health_score baje de cierto umbral o aparezcan hallazgos de severidad alta.

- **Soporte multi-tenant y multi-base.**

Una sola instancia de PgGuardian podrá auditar varias bases de datos o clientes, manteniendo separación lógica entre configuraciones, scans y resultados.

- **Autenticación y control de acceso.**

Se implementará JWT/OAuth y roles básicos para proteger endpoints y restringir acceso a información sensible.

- **Umbrales configurables por cliente.**

Los usuarios podrán ajustar criterios de severidad según tamaño de la base, RAM disponible, workload y criticidad del sistema.

Visión a 1 año:

- **Despliegue en cloud como SaaS.**

Implementar PgGuardian en AWS o GCP como servicio administrado, manteniendo también la opción self-hosted para empresas que no puedan enviar metadata sensible fuera de su infraestructura.

- **Integración con necesidades de cumplimiento para fintechs mexicanas.**

Adaptar los reportes para generar evidencia técnica útil en auditorías, trazabilidad y revisiones internas relacionadas con seguridad, operación y continuidad del servicio.

8. Reparto técnico real

Miembro	Componentes que implementó	Lenguajes/tecnologías que usó
Luis Alberto Arias Llaguno	• Reglas: índices faltantes, duplicados, sequential scans. • API: backend. • Frontend: dashboard	• SQL • Python/FastAPI • React • Docker
Jesús Elias Avila Sauza	• Reglas: índices expresivos, parciales, estadísticas de pg_stats, • API: backend, definición de endpoints.	• SQL • Python/FastAPI • PostgreSQL planner
David Ceballos Mata	• Reglas: conexiones idle, transacciones abiertas, locks, pg_stat_activity • Documento de arquitectura	• SQL • Python • Markdown
Ana Sofía Ruiz Castillo	• Reglas: configuración y memoria: work_mem, shared_buffers, pg_stat_statements.max, log_min_duration_statement, detección de temp_blks_written, queries con alto tiempo acumulado, uso temporal a nivel BD y validación de sort/hash spills con EXPLAIN. • Documento de arquitectura	• PostgreSQL • SQL • Python • Markdown
Isabel Ixchel Torres Sánchez	• Reglas: autovacuum, bloat, filas muertas, tablas que ocupen particionamiento. • Documento de negocio	• SQL • Python

Miembro	Componentes que implementó	Lenguajes/tecnologías que usó
		• PostgreSQL pgstattuple y pg_partman • Investigación de mercado

9. Cómo correr el proyecto (resumen)

Instalación y Configuración

1. Clonar el repositorio

Abrir una terminal en cualquier ruta de su preferencia e ingresar:

```
git clone https://github.com/luiselodeon/PgGuardian.git
```

```
cd PgGuardian
```

2. Crear archivo .env en la raíz del proyecto con los siguientes valores

```
DB_NAME=tiendadb  
DB_USER=tienda_user  
DB_PASSWORD=tienda_pass  
DB_PORT=5433
```

3. Levantar Docker

```
docker compose up -d
```

4. Ligas de acceso

Backend API: <http://localhost:8000/docs>

Frontend: <http://localhost:5173>

Landing page: <https://landing-page-pg-guardian.vercel.app/>

Producto, mercado, modelo de negocio y go-to-market

Documento de Negocio

¿Qué es esto?

Es el documento que evalúa el Criterio 3 (Visión de producto y negocio, 20 pts).

Súbelo al repositorio como /business/negocio.md o /business/negocio.docx.

Extensión recomendada: 8-12 páginas. Más corto = falta análisis. Más largo = no saben qué es importante.

Comprador imaginario de este documento: un inversionista evaluando si invertir en su startup.

1. Resumen ejecutivo

PgGuardian es un producto B2B de auditoría continua para PostgreSQL en producción. Se conecta únicamente de forma de lectura a la base de datos del cliente y entrega un reporte priorizado con hallazgos accionables en cinco categorías: índices, bloat, queries problemáticas, configuración y salud general.

El producto está dirigido a empresas medianas mexicanas y latinoamericanas que tienen Postgres en producción, pero carecen de un DBA dedicado. Estos equipos típicamente tienen 5-50 desarrolladores, una o dos bases de datos críticas, y resuelven problemas de performance reactivamente cuando ya impactaron al cliente.

El competidor más fuerte, pganalyze, cobra USD 199-499/mes y está optimizado para empresas norteamericanas con stacks complejos. Datadog Database Monitoring cobra USD 70/host/mes pero requiere adoptar todo el ecosistema Datadog. PgGuardian se posiciona como opción accesible (USD 49/mes por instancia), localizada (interfaz y soporte en español, recomendaciones que consideran regulación local), y self-hosted opcional (importante para fintechs que no pueden enviar metadata de su BD a un proveedor en el extranjero).

2. Problema

2.1 Descripción del problema

Cuando un equipo de desarrollo construye un producto sobre PostgreSQL, al principio todo parece funcionar a la perfección porque las tablas son pequeñas, las consultas responden en milisegundos y los índices más evidentes están en su lugar. Sin embargo, esta eficiencia inicial es engañosa. Conforme el producto escala y el volumen de datos aumenta, empiezan a acumularse problemas estructurales y de configuración que no son visibles a simple vista, pero que degradan silenciosamente la salud del sistema.

En el ecosistema de las empresas medianas, los desarrolladores suelen estar enfocados en la entrega de nuevas funcionalidades, dejando de lado el ajuste fino de la base de datos tras el despliegue inicial. Al no contar con un DBA dedicado, surgen ineficiencias críticas, como foreign keys sin índices de soporte que fuerzan Sequential Scans masivos en cada JOIN, índices duplicados

que ralentizan las escrituras sin aportar valor real, y tablas con un bloat tan severo que llegan a ocupar hasta tres veces el espacio necesario. A esto se suma el uso de configuraciones por defecto, como un `shared_buffers` limitados a 128 MB en servidores con gigabytes de RAM, lo que asfixia el rendimiento del motor.

Sin una auditoría continua, estos "anti-patrones" permanecen ocultos hasta que alcanzan un punto de ruptura. El resultado es inevitable con consultas que crecen sin control hasta derribar la aplicación a las 3:00 AM, lo que provoca downtime, pérdida de transacciones y un equipo de ingeniería agotado. PgGuardian nace para resolver este vacío, ofreciendo la mirada experta de un consultor automatizado que detecta estos riesgos antes de que se conviertan en una crisis de producción.

2.2 User persona principal

Atributo	Valor
Nombre y rol	Juan Carlos Bodoque, Lead Backend Developer.
Edad y experiencia	30 años; nivel senior/lead con experiencia en desarrollo backend y gestión de infraestructura
Tamaño de empresa	Startup en crecimiento con 40 colaboradores.
Industria	Logística y cadena de suministro (Supply Chain & Logistics).
Día típico de trabajo	Desarrollo de nuevas funcionalidades (features) y atención de emergencias técnicas/operativas.
Pain points principales	Falta de visibilidad proactiva de la salud de la BD; debe realizar diagnósticos manuales complejos en Postgres ante fallos.
Cómo resuelve hoy el problema	Ejecuta consultas manuales directamente al catálogo de Postgres cuando detecta anomalías.
Cuánto tiempo / dinero le cuesta	5 horas semanales de productividad técnica pérdidas; caídas críticas de 4 horas que retrasan las entregas programadas.

Juan Carlos Bodoque, 30 años, es el Lead Backend Developer en una startup logística de 40 personas. Sin un DBA en el equipo, Juan Carlos reparte su jornada entre el desarrollo de features y la atención de emergencias de base de datos que el mismo debe diagnosticar mediante consultas manuales al catálogo de Postgres. Esta gestión reactiva le consume cerca de 5 horas a la semana y ya ha provocado caídas críticas de 4 horas durante picos de operación, afectando seriamente las entregas programadas. Juan Carlos necesita una herramienta que le brinde auditoría continua y recomendaciones listas para ejecutar, permitiéndole recuperar su tiempo de desarrollo y asegurar la estabilidad de la plataforma sin añadir complejidad técnica

2.3 Frecuencia y severidad

La frecuencia del problema en este sector es constante debido al flujo ininterrumpido de datos de rastreo e inventario, lo que genera problemas crónicos de bloat e índices. Sin un DBA, se presentan incidentes agudos de una a dos veces por trimestre. La severidad es crítica, ya que una caída de 4 horas en picos de operación retrasa las entregas programadas, afectando directamente la promesa de servicio al cliente. El costo se estima mediante el siguiente razonamiento: Juan Carlos, con un costo laboral de \$30 USD/hora, pierde 5 horas semanales en diagnósticos manuales. Esto representa una fuga de \$7,800 USD anuales en productividad ($\$30 * 5 \text{ horas} * 52 \text{ semanas}$).

3. Investigación de usuarios

Requisito obligatorio

Mínimo 3 entrevistas con personas del rol objetivo. Si no las hicieron, este criterio se va a 0.
Las entrevistas pueden ser de 30 minutos, por video o presenciales.
Documenten preguntas hechas, respuestas resumidas, y aprendizajes.

3.1 Resumen de entrevistas

Nombre / Rol del entrevistado	Empresa / Sector	Fecha	Duración
Edmundo Humberto Torres Cruz / Project Manager	MetLife / Seguros	08/05/2026	14:47 min
Christina Bautista / DBA	Vintech / Consultoría	13/05/2026	16:12 min
Sergio Carrion Aburto / DBA	Bintech S.A. Enterprise / Consultoría	11/05/2026	39:04 min

3.2 Preguntas hechas

- ¿Quién se encarga normalmente de revisar el rendimiento y la salud de sus bases de datos PostgreSQL?
- En los últimos meses, ¿han tenido algún incidente relacionado con base de datos, como lentitud, caídas, bloqueos, queries pesadas o problemas de espacio? ¿Cuánto tiempo les tomó resolverlo?
- ¿Cómo detectan actualmente que una consulta está lenta, que falta un índice o que la base empieza a degradarse: por monitoreo interno, alertas, revisión manual o reportes de usuarios?
- Si recibieran un reporte semanal con los tres problemas más importantes de la base de datos y recomendaciones concretas para corregirlos, ¿qué valor tendría para su equipo?

5. ¿Han usado o evaluado herramientas como pganalyze, Datadog Database Monitoring, pgwatch2, Percona u otra herramienta de monitoreo de bases de datos? ¿Qué les gustó o qué les impidió adoptarla?
6. ¿Qué tan importante sería para ustedes que la herramienta tuviera reportes en español, soporte local o una opción self-hosted para no enviar información sensible fuera de su infraestructura?
7. Cuando necesitan revisar o mejorar una base de datos, ¿prefieren resolverlo internamente, contratar consultoría puntual o pagar una herramienta mensual? ¿Por qué?
8. Para que una prueba de 14 días valga la pena, ¿qué resultado concreto tendría que mostrarles la herramienta? Por ejemplo: horas ahorradas, queries detectadas, reducción de riesgo, reporte para gerencia o recomendaciones aplicables.

3.3 Aprendizajes principales

- **Aprendizaje 1. El monitoreo debe ir más allá de queries lentas**
Los problemas reales en producción frecuentemente se originan en conexiones mal manejadas, respaldos mal configurados, procesos externos o saturación de infraestructura. PgGuardian debe evolucionar para contextualizar mejor la causa raíz, no solo señalar síntomas.
- **Aprendizaje 2. El análisis de logs tiene alto valor diagnóstico**
El entrevistado confirmó que los logs de PostgreSQL permiten detectar procesos internos, cascadas de consultas y anomalías que un monitoreo superficial no alcanza a capturar. Incorporar análisis básico de logs sería un diferenciador importante.
- **Aprendizaje 3. Las métricas de memoria y spills sí son relevantes**
Se validó que operaciones como work_mem, escritura a disco por sorts, índices mal diseñados y operaciones temporales afectan directamente el rendimiento.
- **Aprendizaje 4. Las recomendaciones deben ser accionables, no solo alertas**
La herramienta debe sugerir posibles causas y comandos SQL concretos para resolver cada problema. Esto acelera la corrección y reduce la fricción para equipos sin DBA.
- **Aprendizaje 5. El mercado ideal son equipos sin DBA especializado**
PgGuardian tendría mayor impacto en empresas pequeñas y medianas que tienen PostgreSQL en producción, pero que carecen de un administrador dedicado.
- **Aprendizaje 6. El soporte en español es un diferenciador** PgGuardian se volvería una herramienta de mitigación de crisis para competidores extranjeros que se desempeñan en LATAM, ya que tener un soporte nativo en español permitiría mejorar mucho el problema de raíz.
- **Aprendizaje 7. Validación del Self-hosted** ya que se podría estar manejando información confidencial de clientes y se debe de garantizar esta seguridad y cifrado. Con la opción de Self-hosted las empresas no tendrían por qué mandar su metadata a un SaaS externo.
- **Aprendizaje 8. Validación de la generación de reportes continuos** los reportes diarios sobre índices faltantes o consultas permite a los Lead Developers a justificar porque se necesita tiempo para optimizar la base de datos y no solo programar nuevas funciones.
- **Aprendizaje 9. Prueba gratuita** nos recomiendan una prueba gratuita de un mes con reportes comparativos con respecto al mes anterior dónde se puede identificar que usuario ejecuta queries más pesadas, intentos de acceso, consultas problemáticas, uso excesivo de tablas, picos en el CPU/memoria, consumo de disco y que funcione en ambientes híbridos (físicos y nube).
- **Aprendizaje 10. La facilidad de configuración es clave para la adopción**, para que una herramienta como PgGuardian sea usada realmente, no basta con que detecte problemas

técnicos. También debe ser fácil de parametrizar, configurar y entender. Los reportes y alertas deben ser claros, porque si la herramienta es compleja de interpretar, el equipo puede abandonarla, aunque técnicamente sea útil.

3.4 Resumen de entrevistas

Entrevista: Edmundo Torres

La entrevista se realizó con Edmundo Torres Scrum Master profesional de TI con maestría en Sistemas, actualmente desempeña como Project Manager especializado en migraciones y desarrollo de sistemas.

En la entrevista se menciona que la empresa como trata con datos sensibles, las políticas de las bases de datos deben estar muy reguladas. Además de que actualmente la empresa tiene un soporte 24/7 con soporte local en México durante el día y soporte offshore en India después de las 5:30 p.m. hora México.

Además, nos comenta que actualmente utilizan monitoreos proactivos usando Tivoli y MATLAB con IA para detectar problemas como bloqueos de sesión y consultas pesadas. Estos reportes se utilizan para evaluar el costo/beneficio de la optimización del espacio y cómo planificar mejoras en el backlog cada 10 semanas.

Highlights principales:

- Un error crítico puede llegar a ser costoso. Nos contó que sufrieron SQL injection dónde se borró una tabla de 23 millones de registros, lo que saturó el disco por exceso de lectura y escritura provocando una caída total del sistema de 9 horas.
- El tener un soporte en español resulta muy útil pues a veces existen barreras en la comunicación.
- Están incorporando el uso de IA, pues ya no se hace el monitoreo 100% manual.
- Al ser una empresa que maneja datos sensibles cualquier herramienta externa debe pasar filtros de seguridad y cifrado.

Link a la grabación:

https://www.youtube.com/watch?v=RgaZixuk_ag

Entrevista: Sergio Carrion

La entrevista se realizó con Sergio Carrión, subdirector de bases de datos con aproximadamente 20 años de experiencia trabajando con tecnologías como PostgreSQL, Oracle, SQL Server y SAP Hana. Durante la conversación se validaron distintos problemas reales relacionados con monitoreo, rendimiento y operación de bases de datos PostgreSQL en ambientes productivos.

El entrevistado explicó que los problemas de rendimiento no siempre provienen directamente de la base de datos, sino que muchas veces están relacionados con infraestructura, red, concurrencia de usuarios, seguridad o servidores. También comentó que su equipo controla completamente los queries utilizados en producción, realizando pruebas y optimización antes de liberar cualquier consulta.

Además, mencionó que las herramientas de monitoreo existentes son útiles principalmente cuando no existe un equipo DBA consolidado, aunque reconoció valor en soluciones que ayuden a detectar causas raíz, problemas de memoria, conexiones excesivas o comportamiento anormal de usuarios.

También se destacó la importancia del análisis de logs para identificar problemas complejos, así como la necesidad de generar comparativos históricos y recomendaciones accionables para demostrar el valor real de una herramienta de monitoreo.

Highlights principales:

- Los problemas de rendimiento frecuentemente están relacionados con infraestructura y concurrencia, no únicamente con queries SQL.
- El equipo DBA controla completamente las consultas liberadas a producción.
- El análisis de logs fue identificado como una de las herramientas más útiles para diagnóstico avanzado.
- Se validó la importancia de detectar:
 - uso excesivo de memoria, operaciones temporales, saturación de conexiones y comportamiento anormal de usuarios.
- El entrevistado consideró útil recibir recomendaciones concretas y explicaciones sobre la causa de los problemas.
- Una prueba corta de 14 días fue considerada insuficiente para validar una herramienta de monitoreo de bases de datos.

Link a la grabación:

<https://www.youtube.com/watch?v=78VRi8cbvN4>

Entrevista: Cristina Bautista

La entrevista se realizó con Cristina Bautista, profesional con experiencia en bases de datos tanto en gobierno como en iniciativa privada. Durante la conversación comentó que ha trabajado en instituciones como la Secretaría de Finanzas y el Instituto Federal de Telecomunicaciones, además de participar actualmente en soluciones relacionadas con bases de datos en el sector privado. Aunque su experiencia reciente incluye SAP HANA, también mencionó experiencia previa con PostgreSQL, destacando que muchos principios de SQL y administración de bases de datos son aplicables entre distintas tecnologías.

La entrevistada explicó que la responsabilidad de revisar el rendimiento y la salud de las bases de datos normalmente recae en equipos de infraestructura y administradores de bases de datos. Sin embargo, en empresas más pequeñas esta tarea puede terminar en manos de los mismos desarrolladores, especialmente cuando no existe un equipo DBA dedicado. Esto valida directamente el problema que busca atender PgGuardian: apoyar a equipos que necesitan visibilidad sobre su base de datos, pero que no siempre cuentan con especialistas dedicados.

También comentó que los incidentes relacionados con bases de datos son comunes, incluyendo bloqueos, crecimiento inesperado de espacio, lentitud en consultas y degradación del sistema. Dependiendo de la gravedad, algunos problemas pueden resolverse en minutos u horas, pero otros pueden tomar uno o dos días cuando requieren intervención del equipo de infraestructura. Además,

señaló que actualmente muchas veces los problemas se detectan por reportes de usuarios o porque el equipo nota que la base comienza a responder de forma anormal, más que por alertas preventivas automáticas.

Respecto al valor de una herramienta como PgGuardian, la entrevistada destacó que sería útil contar con monitoreo automatizado, reportes claros, alertas configurables y recomendaciones que ayuden a prevenir incidentes antes de que se conviertan en problemas graves. También mencionó que el costo suele ser una barrera importante para adoptar herramientas de monitoreo, especialmente en gobierno, por lo que una solución accesible, en español y con posibilidad de soporte local o self-hosting podría facilitar su adopción en equipos mexicanos.

Finalmente, la entrevistada señaló que para considerar el uso de una herramienta nueva sería importante realizar una prueba de concepto gratuita, donde se demuestre claramente qué detecta, qué reportes genera y qué riesgos identifica. También consideró relevante que la herramienta sea fácil de configurar, que sus reportes sean entendibles y que pueda mostrar información como consumo de memoria, carga de la base, usuarios conectados, queries en ejecución, picos anormales y consumo de procesamiento.

Highlights principales:

- En empresas pequeñas, el monitoreo puede recaer en desarrolladores cuando no existe un DBA dedicado.
- Actualmente, muchos problemas se detectan de forma reactiva: por reportes de usuarios o porque el equipo nota comportamientos anormales.
- Se validó la necesidad de monitoreo automatizado, alertas configurables y reportes claros.
- El costo fue identificado como una de las principales barreras para adoptar herramientas de monitoreo.
- Se consideró valioso detectar consumo de memoria, carga de la base de datos, queries pesadas, usuarios conectados, picos anormales y consumo de procesamiento.
- Una prueba de concepto gratuita fue mencionada como un factor clave para generar confianza y evaluar si la herramienta realmente aporta valor.

Link: [Entrevista Cristina PgGuardian](#)

4. Solución

Desde la perspectiva de Juan Carlos Bodoque, PgGuardian se integra a su flujo de trabajo no como una carga técnica adicional, sino como un colaborador experto que elimina la incertidumbre operativa. Al utilizar el producto por primera vez, Juan Carlos se encuentra con un proceso de configuración simplificado que respeta la seguridad de su infraestructura logística. En menos de cinco minutos, registra una conexión read-only (URL de conexión y usuario sin permisos de escritura) desde el dashboard de PgGuardian. A diferencia de otras herramientas, no necesita instalar agentes invasivos en sus servidores ni pasar por complejos procesos de aprobación de cambios.

A las 24 horas de la configuración inicial, Juan Carlos recibe el primer reporte de salud accionable, transformando lo que antes era una mañana de diagnósticos manuales en una sesión de toma de

decisiones estratégicas. El sistema ya ha analizado el catálogo de su base de datos y le entrega un reporte priorizado por severidad que incluye evidencia técnica de los problemas detectados. Por ejemplo, en lugar de solo avisar de lentitud, PgGuardian le muestra la consulta exacta que está causando un Sequential Scan en su tabla de envíos y le proporciona el código SQL listo para crear el índice faltante.

Con el uso continuo, el valor del producto se potencia mediante la comparación temporal. Juan Carlos puede visualizar si las optimizaciones aplicadas realmente redujeron el bloat o mejoraron el rendimiento respecto al escaneo anterior. Esto le permite recuperar sus cinco horas semanales de gestión reactiva para enfocarse en el desarrollo de features críticas para la logística del startup.

4.1 Funcionalidades core

Feature	Beneficio para el usuario
Índices inteligentes	Detecta índices duplicados, índices no usados que solo consumen espacio, índices faltantes con llaves foráneas. Sugiere la creación de índices en tablas donde se tenga datos grandes y un solo valor domine la mayoría de los registros y en tablas donde se recomienda usar Covering Index. Busca estadísticas obsoletas, al usarlo el usuario ahorra horas de análisis manual.
Control de bloat	Calcula el porcentaje de espacio desperdiciado en tablas debido a filas muertas. Verifica, tablas donde se tenga el AUTOVACUUM desactivado. El usuario recupera almacenamiento y evita que las consultas masivas se vuelvan lentas por escanear páginas vacías.
Optimización de memoria	Identifica queries que tienen un work_mem bajo, la evaluación de uso de archivos temporales, validación de configuración contra workload observado y la generación de hallazgos y recomendaciones. Reduce latencia en operaciones complejas
Monitoreo de conexiones problemáticas	Identifica conexiones idle in transaction, conexiones abandonadas, transacciones largas que retienen locks y tablas que necesiten de particionamiento siguiendo un conjunto de características predefinidas. Ajusta timeouts antes de que derriben la base de datos.
Reporte	Entrega un reporte con los problemas que se enfrenta la base de datos analizadas con los features antes mencionados. Incluye un dashboard donde se ve el estado de salud en el que se encuentra la BD y adicional cada uno de nuestros features tiene un apartado en donde se puede ver con más detalles los posibles hallazgos de problemas.

5. Análisis competitivo

Requisito obligatorio

Mínimo 3 competidores reales investigados. "No hay competencia" = 0 pts automático.

Investiguen precios, features y segmento objetivo de cada uno.

El espacio en blanco es lo más importante: dónde están débiles vs ustedes.

5.1 Tabla comparativa

Característica	pganalyze	Datadog DBM	pgwatch2	PgGuardian
Precio Mensual Base	USD 149 (~2560 MXN)	USD 23/host (~400 MXN)	Gratis (OSS)	\$850 MXN
Tier inicial accesible	No	Si	Si (Gratis)	Si (Enfoque PyME)
Self-hosted	Solo Enterprise	No	Sí	Sí (Docker Compose)
Soporte en español	No	Limitado	No	Sí (Nativo)
Foco LATAM / Regulación	No	No	No	Sí (Prioritario)
Setup	Requiere Agente	Agente Datadog	Complejo (OSS)	Zero-install (Read-only)
Modelo de negocio	SaaS B2B	SaaS (Stack DD)	Open Source	SaaS + Licencia

5.2 Análisis honesto

pganalyze

- **Mejor que nosotros en:** cobertura técnica más amplia (10+ años de desarrollo), integración con stacks complejos, comunidad establecida.
- **Nosotros somos mejores en:** precio accesible para empresas medianas LATAM, soporte en español, foco en regulación local.

Datadog Database Monitoring

- **Mejor que nosotros en:** ecosistema completo (logs, APM, métricas en un solo dashboard), red de soporte global.
- **Nosotros somos mejores en:** no requiere adoptar todo Datadog, no envía metadata sensible de la BD a servidores en el extranjero.

pgwatch2

- **Mejor que nosotros en:** precio (gratis), totalmente open source.
- **Nosotros somos mejores en:** producto terminado vs. herramienta de monitoreo. PgGuardian dice qué arreglar; pgwatch2 solo grafica métricas que el cliente debe interpretar.

5.3 Espacio en blanco

El nicho estratégico de PgGuardian se centra en empresas mexicanas en etapa de crecimiento, como startups de logística y fintechs reguladas, que operan con PostgreSQL en producción, pero carecen de un DBA dedicado para asegurar su estabilidad. Los competidores actuales no logran cubrir este segmento eficazmente: pganalyze maneja precios prohibitivos para el mercado latinoamericano, Datadog exige una adopción integral de su ecosistema que resulta excesiva para equipos pequeños, y pgwatch2 requiere un nivel de experiencia técnica que el cliente precisamente no posee. Frente a esto, nuestra propuesta de valor reside en el profundo conocimiento del contexto regional (abarcando idioma, regulación local y presupuestos realistas) junto con un modelo de negocio accesible de \$850

MXN y la opción de despliegue self-hosted, garantizando así la soberanía de datos sensibles sin comprometer la agilidad operativa.

6. Modelo de negocio

6.1 Tipo de modelo de revenue

- ☒ Suscripción mensual / anual
- ☐ Por uso (pago por análisis ejecutado, etc.)
- ☐ Por seat (por usuario activo)
- ☒ Por instancia (por BD monitoreada)
- ☒ Freemium con tier pago
- ☐ Otro: _____

6.2 Tiers de pricing

Tier	Precio	Comprador objetivo	Qué incluye
Inicial	\$850 MXN/mes	Startups con 1-2 BDs	1 instancia, scans semanales, dashboard, alertas email
Pro	\$2000 MXN/mes	Empresas medianas	Hasta 5 instancias, scans diarios, alertas Slack, soporte 24h
Empresarial	\$4500+ MXN/mes	Fintechs con compliance	Self-hosted, SSO, auditoría custom, soporte 24h

6.3 Justificación del precio

\$850 MXN/mes para el tier Inicial Starter está calibrado para que sea alrededor de 4 veces más barato que pganalyze (~3400 MXN) y por debajo del threshold de "approval de gerencia" en empresas mexicanas (~USD 1000/mes en gastos de SaaS suelen requerir aprobación).

\$2500 MXN/mes para el tier Pro lo decidimos de esta forma porque se posiciona como una alternativa a pganalyze base, con features que ellos tienen, pero localizado y más simple de entrar.

\$4500+ MXN/mes para el tier Empresarial es donde está el revenue real: Fintechs que necesitan self-hosted por compliance pagan esto sin parpadear si el producto les da auditoría continua que justifique frente a CNBV.

7. Tamaño de mercado

TAM / SAM / SOM

TAM (Total Addressable Market): el universo total. Citen fuentes (Gartner, IDC, etc.).

SAM (Serviceable Addressable Market): la porción que pueden atender realísticamente.

SOM (Serviceable Obtainable Market): lo que realmente capturarían en 3-5 años.

7.1 TAM

El Total Addressable Market (TAM) es una métrica que permite evaluar la posible cantidad de ingresos si nuestro producto captara el 100% del mercado.

El TAM se obtiene de la siguiente manera:

$$TAM = \#TOTAL\ CONSUMIDORES * PRECIO\ PROMEDIO\ ANUAL$$

Según los resultados de los Censos Económicos 2024 del INEGI aproximadamente 1.3 millones de unidades económicas utilizan herramientas de software para analizar datos masivos. Tomando en cuenta la suscripción de Datadog DBM, el precio promedio anual es de 276 USD (\$4750 MXN). Ahora, sustituimos en la fórmula los valores quedando de la siguiente manera:

$$TAM = 1,300,000 * \$4,750$$

El cálculo del TAM nos revela que el máximo de ingresos para este servicio. Al proyectar que 1.3 millones de empresas objetivo adquirieran la suscripción anual de \$4,750 MXN, se concluye que en este mercado tiene una oportunidad de \$6,175 millones MXN.

Referencias:

INEGI. (2025, 24 julio). *REPORTE DE RESULTADOS 22/25*.
https://www.inegi.org.mx/contenidos/saladeprensa/boletines/2025/ce/CE2024_def_RR.pdf
Karla Garcia. (2025, 9 noviembre). *Cómo CALCULAR el Tamaño de tu MERCADO OBJETIVO: TAM, SAM y SOM* [Video]. YouTube. <https://www.youtube.com/watch?v=EuxJNBqkeAk>
TAM SAM SOM. (2026, 11 mayo). [Video]. Amazon Ads.
<https://advertising.amazon.com/es-mx/library/guides/tam-sam-som#2>

7.2 SAM

El Serviceable Addressable Market (SAM) es la porción del TAM que es relevante para nuestro negocio.

El SAM se obtiene de la siguiente manera:

$$SAM = TAM * \%CONSUMIDORES\ QUE\ SE\ AJUSTAN\ A\ TU\ PRODUCTO$$

Nuestro foco estaría más centrado en startups, empresas logísticas y el sector Fintech. De acuerdo con datos del ecosistema Fintech de México recopilados por Finnovista Fintech Radar hay 1,100 Fintech operando. Con este dato, sustituimos los valores de la fórmula quedando:

$$SAM = \$6,175,000,000 * 0.2\%$$

El cálculo del SAM nos da una visión más aterrizada sobre el negocio, concluyendo que tenemos un mercado accesible de \$12.35 millones MXN.

Referencias:

Calleja, P. D. (2025, 27 marzo). *Finnovista Fintech Radar lanzó su reporte sobre el ecosistema Fintech de México. Checa estos datos: - Más de 1,100 Fintech operando - Incremento de 31% en ingresos (¡las scale-ups van con todo!) - . . . | Patricia Desentis Calleja* [Video]. LinkedIn.
https://www.linkedin.com/posts/patricia-desentis_finnovista-fintech-radar-lanz%C3%B3-su-reporte-ugcPost-7310872211211366400-R4w1/
Karla Garcia. (2025, 9 noviembre). *Cómo CALCULAR el Tamaño de tu MERCADO OBJETIVO: TAM, SAM y SOM* [Video]. YouTube. <https://www.youtube.com/watch?v=EuxJNBqkeAk>

TAM SAM SOM. (2026, 11 mayo). [Vídeo]. Amazon Ads.
<https://advertising.amazon.com/es-mx/library/guides/tam-sam-som#2>

7.3 SOM

El Serviceable Obtainable Market (SOM) es la porción del SAM que nuestro producto puede captar.

El SOM se obtiene de la siguiente manera:

$$SOM = SAM * \%MERCADO QUE PUEDES CAPTURAR$$

Nuestra definición de mercado a 3-5 años sería capturar el 0.2% del mercado accesible (SAM).
Asumiendo esto sustituimos los valores en la fórmula:

$$SOM = 1,100 \text{ fintechs} \times 5\% = 55 \text{ clientes}$$
$$\text{Revenue SOM} = 55 \times \$850 \text{ MXN/mes} \times 12 = \$561,000 \text{ MXN/año}$$

Este resultado quiere decir que queremos alcanzar aproximadamente a 55 empresas activas mediante un enfoque estratégico de nicho, esto significa ventas directas a CTOs y/o líderes técnicos en hubs tecnológicos.

Referencias:

Karla Garcia. (2025, 9 noviembre). *Cómo CALCULAR el Tamaño de tu MERCADO OBJETIVO: TAM, SAM y SOM* [Vídeo]. YouTube. <https://www.youtube.com/watch?v=EuxJNBqkeAk>

TAM SAM SOM. (2026, 11 mayo). [Vídeo]. Amazon Ads.
<https://advertising.amazon.com/es-mx/library/guides/tam-sam-som#2>

7.4 Tendencias relevantes

- **Automatización y IA:** El uso de herramientas avanzadas de automatización e inteligencia artificial puede automatizar tareas como la optimización de consultas, detectar anomalías, reparar problemas lo que reduce la carga de trabajo manual y mejora la eficiencia.
- **BD en la nube:** Migración a bases de datos basadas en la nube. AWS, Azure y Google Cloud ofrecen sus servicios permitiendo una escalabilidad dinámica, redundancia mejorada y reducción de costos operativos.
- **BD Multi-Modelo:** Este tipo de base de datos maneja diferentes tipos de datos (relaciones, gráficos...) en un solo sistema, permitiendo una flexibilidad significativa para las organizaciones que manejan diferentes tipos de datos.

Referencias:

Mesanza, F. (2024, 24 mayo). *Principales 10 tendencias en la gestión de bases de datos: Qué deben saber los directores de TI en 2024*. KIT Ingeniería Electrónica.
<https://www.kit.com.ar/principales-10-tendencias-en-la-gestion-de-bases-de-datos-que-deben-saber-los-directores-de-ti-en-2024/>

8. Go-to-market

3.3 Primeros 10 clientes

- **PASO 1 (SEMANAS 1-3):** Mapeo. Identificar 20 startups logísticas o Fintech en el ecosistema emprendedor regional (Vison Hub, Parque Tecnológico Innovación Querétaro, BLOQUE, Parque Tecnológico Ciudad Maderas o el Parque Tecnológico del Tec de Monterrey) con el fin de contactar a los Lead Developers o CTOs a través de LinkedIn con un mensaje directo. O asistir a eventos de Networking para conectar con estos perfiles.
- **PASO 2 (SEMANAS 4-6):** Auditorías. Ofrecer el servicio de PgGuardian sin costo en un entorno emprendedor. Se puede ofrecer la prueba gratuita donde se pueda observar el reporte con todas las incidencias marcadas, demostrando queries lentas o índices faltantes que les están haciendo perder dinero.
- **PASO 3 (SEMANAS 7-10):** Publicar un artículo técnico en Medium o LinkedIn con el nombre de *Los 5 problemas ocultos en PostgreSQL que matan tu startup (y cómo detectarlos gratis)*. Y ofrecer una auditoría gratuita a las primeras 10 startups que llenan un formulario proporcionado en el artículo.
- **PASO 4 (SEMANAS 11-13):** Identificar 10 startups con el perfil objetivo en Startup México, LinkedIn o redes sociales. Enviar un correo personalizado a CTOs o Lead Backend Developers ofreciendo el primer mes gratis.
- **PASO 5:** Asistir a Talent Land México, uno de los eventos más grandes sobre emprendimiento e innovación tecnológica. Llevar la demo lista y ofrecer pruebas gratis en el evento.

Se tiene contemplado un timeline de 4 meses aproximadamente desde el lanzamiento oficial para conseguir 10 clientes. Aunque, se podría conseguir más clientes asistiendo a Talent Land México que será del 30 de marzo al 1 de abril de 2027.

Costos asociados:

- **Anuncios:** Utilizando Google Ads ocuparíamos una inversión de \$5,000 MXN al mes.
- **Tiempo del equipo:** Aproximadamente 6 horas por semana.
- **Costos de entrada a eventos de networking:** Aproximadamente \$3,000 MXN.

Referencias:

Kabana. (2026, 15 abril). *¿Cuánto cuesta la publicidad digital en México en 2026? Guía completa*. Agencia de Marketing Kabana. <https://kabanamx.com/cuanto-cuesta-publicidad-digital-mexico/>

Talent Land. (s. f.). <https://mx.talent-land.com/>

3.4 Estrategia de growth (después de los 10)

- **Canal 1 (Contenido técnico real):** Publicar en Medium o LinkedIn casos de éxito anónimos como *Cómo redujimos el bloat en un 60% en una BD de logs*.
- **Canal 2 (Referidos):** Crear una campaña con nuestros clientes actuales donde por cada cliente referido el cliente reciba un mes gratis y así crecer nuestros usuarios.
- **Canal 3 (Partners):** Juntarnos con bootcamps o consultoras pequeñas de México para que recomienden el uso de PgGuardian a sus clientes y cobrar una pequeña comisión.
- **Canal 4 (Pruebas gratis):** Crear un sitio web con un dashboard público implementando la demo, donde los clientes puedan probar su funcionamiento y al terminar la prueba, mostrar qué tipo de problemas encontrarían si siguieran utilizando PgGuardian.

9. Diferenciador defendible

La pregunta de oro

Si el competidor más fuerte les copia el feature mañana, ¿por qué los clientes seguirían con ustedes?

Si la respuesta es "somos más baratos", están en problemas: te copian el precio.

Diferenciadores defendibles: regulación local, vertical específico, distribución única, modelo distinto.

No solo somos más baratos. Somos el único producto de auditoría PostgreSQL construido específicamente para startups y medianas empresas latinoamericanas que no tienen DBA, con idioma, contexto regulatorio (Fintechs mexicanas) y un modelo self-hosted como opción nativa.

Es defendible con el tiempo porque tenemos un conocimiento del mercado mexicano:

- Contamos con una regulación local en base a las Fintechs.
- Nuestros costos están basados en presupuestos reales de PyMEs mexicanas.
- Contamos con soporte en español y la documentación será extensa pero enfocada a ejemplos.
- Nuestros competidores no adaptan sus precios para la comunidad de LATAM, nosotros en cambio, estamos enfocados en nuestros clientes de LATAM.
- El enfoque iría en PyMEs que no tienen un área designada para el manejo de bases de datos.

Al hacer nuestro análisis de clientes potenciales, un cliente Fintech nos elige sobre pganalyze porque puede desplegar PgGuardian self-hosted dentro de su VPC (Virtual Private Cloud), cumpliendo con la normativa de datos pertinente y recibe reportes en español utilizando las leyes locales y paga \$850 MXN al mes.

10. Por qué nuestro equipo

Somos un equipo que cubrimos todo el espectro: quien construye la API, quien diseña el frontend, quien documenta la arquitectura, quien analiza el mercado y cada uno se enfocó en un área crítica de PostgreSQL. Hablamos con profesionales del sector y obtuvimos aprendizajes concretos como:

- Análisis de logs como requerimiento real.
- Comparativas mensuales y trazabilidad por usuario.
- Recomendaciones accionables.
- Enfoque en PyMEs sin DBA.

Construyendo así un producto enfocado al usuario, además del Vision Hub nos puede ayudar a conseguir contactos para lanzar el producto. Al mismo tiempo PgGuardian ya se conecta a PostgreSQL en modo read-only, ejecuta reglas de detección y genera reportes. Este MVP fue probado con la BD demo TiendaDB y obtuvimos métricas reales de los problemas volviendo en un producto que ya funciona.

11. Roadmap a 12 meses

Trimestre	Producto	Negocio	Operativo
Q1 (0-3 meses)	MVP estable con identificación de problemas relacionados a índices, bloat, queries y configuración. Incluye dashboard básico y exportación PDF.	Conseguir a los primeros 10 clientes de pago (plan empresarial). Landing page y vídeo del demo funcional.	Docker Compose funcional. README probado por un externo.

Trimestre	Producto	Negocio	Operativo
Q2 (4-6 meses)	Añadir más posibilidades de problemas. Crear alertas personalizadas por correo. Documentar el modo self-hosted.	Lanzar prueba gratuita de un mes, para poder realizar el comparativo del mes siguiente. Publicar 3 casos de éxito anónimos y conseguir nuestro primer referido.	Contratar una persona para soporte técnico.
Q3 (7-9 meses)	Dashboard con comparación histórica. Integrarlo con softwares de gestión de empresa como Slack, ClickUp...	Llegar a 20 clientes pago y buscar una alianza con una consultora pequeña de Querétaro, México.	Automatizar pagos y facturación.
Q4 (10-12 meses)	Documentar el uso de normas mexicanas para Fintechs mexicanas (por ejemplo, uso de información sensible).	Buscar una inversión semilla de \$1,000,000 MXN para escalar ventas.	Contratar un representante de ventas.

12. Ask

Para alcanzar el siguiente nivel necesitamos:

1. Una inversión semilla de \$100,000 MXN para contratar a los desarrolladores necesarios para mantener la PgGuardian y asistir a eventos de networking clave.
2. Tener contacto con expertos que tengan experiencia en escalamiento de SaaS B2B y en go-to-market.
3. Buscar cinco clientes pilotos para validar el precio y las features con 90 días de prueba gratuita a cambio del testimonio.
4. Tener acceso a inversionistas ángel con foco en software B2B LATAM.

Antes de entregar el documento de negocio

1. Verifiquen que mencionaron mínimo 3 competidores reales con datos verificables.
2. Confirman que las entrevistas existen (el profesor puede pedir contactos).
3. TAM/SAM/SOM tienen fuentes citadas.
4. El diferenciador no es solo "más barato".
5. El plan de primeros 10 clientes es ejecutable, no genérico.

— Fin del documento —